CISC 3130 Homework: Build a Music Player

Oct 7, 2026 · @Steven

Overview

You will build the engine behind a music player: a playlist, an "up next" list, a back button and instant song lookup. No single data structure does all of that well, so your player will combine five of them and make them work together.

Due: [due date] · **Points:** 100 (+10 extra credit) · **Work:** individual

By the end you should be able to:

- Choose the right data structure for a job and defend the choice with Big-O.
- Make several structures cooperate inside one class without letting them fall out of sync.
- Explain your design the way an engineer would in an interview.

Rules

Language. Use JavaScript or Java. Pick one. The starter folder has both.

Build or reuse. You choose how each structure is made. Both paths can earn full credit.

Path	What it means	JavaScript	Java
Build	You write the structure yourself from linked nodes	Your own `DNode`, `Deque`, `Stack` classes	Your own `Node` class and linked structures
Reuse	You use what the language gives you	Arrays and `Map`	`LinkedList`, `ArrayDeque`, `HashMap`

You may mix paths, for example build the playlist and reuse the rest.

Hash map required. Your player must use a hash map for song lookup. Use `Map` in JavaScript or `HashMap` in Java. You do not have to build the hash map yourself.

Tell the truth about cost. Reusing is fine, but some built-ins are slow for this job. A JavaScript array's `shift()` and `unshift()` are O(n). Your Big-O table must report what your code really costs, not what the ideal structure would cost.

Do not change the method names, parameters or return types in the starter file. The test script depends on them.

The system

Your `MusicPlayer` class holds five structures and one variable, `current`, for the song playing now.

Part	Job	Suggested structure	Why
Playlist	Keep songs in order; move to the next song	Doubly linked list	O(1) to step forward or remove a node you already hold
Library	Find any song by id	Hash map (id → the song's place in the playlist)	O(1) lookup with no scanning
Up next	Songs waiting to play; add at the front or the back	Deque	O(1) at both ends
History	Songs already played; the back button	Stack	The most recent song comes off first
Play counts	How many times each song has played	Hash map (id → count)	O(1) update

Methods you must write

Method	What it does	Returns
`addSong(id, title, artist)`	Adds a song to the end of the playlist and to the library	`true`, or `false` if the id already exists
`findSong(id)`	Looks up a song in O(1)	The song, or `null`
`showPlaylist()`	Lists titles in playlist order	List of titles
`play(id)`	Starts a song	The song, or `null` if the id is unknown
`nowPlaying()`	Reports the current song (already written for you)	The song, or `null`
`playCount(id)`	Reports how many times a song has played	A number; 0 if never
`queueLater(id)`	Adds a song to the back of up next	`true`, or `false` if the id is unknown
`queueNext(id)`	Adds a song to the front of up next	`true`, or `false` if the id is unknown
`showUpNext()`	Lists up next, front first	List of titles
`next()`	Moves to the next song	The song, or `null` if there is none

Method	What it does	Returns
`back()`	Returns to the previous song	The song, or `null` if history is empty
`showHistory()`	Lists history, most recent first	List of titles
`removeSong(id)`	Removes a song from the playlist and library	`true`, or `false` if unknown or playing now

How the parts work together

These six rules are the assignment. Each one touches more than one structure.

1. **A song becomes current.** Whenever a song starts, by `play`, `next` or `back`, its play count goes up by 1.
2. **`play(id)`.** If a song was already playing, push it onto history first. Then the new song becomes current.
3. **`next()`.** Take the front of up next if it has a song. If up next is empty, take the song after the current one in the playlist. If there is no such song, return `null` and change nothing. Otherwise push the old current song onto history.
4. **`back()`.** Pop the most recent song from history and make it current. The song that was playing goes to the front of up next, so `next()` brings it right back. `back()` does not push anything onto history.
5. **`removeSong(id)`.** Remove the song from the playlist and the library, and forget its play count. You cannot remove the song that is playing now.
6. **Removed songs are skipped.** A removed song may still be sitting in up next or history. `next()` and `back()` must skip it, and `showUpNext()` and `showHistory()` must not list it.

Steps

Build one feature at a time and run the test script after each step. The step numbers match the comments in the starter file and the sections of the test output.

Step 0. Set up.

- Open the `javascript` or `java` folder from the starter files.
- Run the test script before writing anything, so you know it runs. JavaScript: `node test.js`. Java: `javac *.java && java Test`.
- Decide build or reuse for each of the five structures. Write your choices in a comment at the top of your file.
- Create the five structures in the constructor (JavaScript) or as fields (Java).

Step 1. Playlist and library. Write `addSong`, `findSong` and `showPlaylist`.

- `addSong` must update two structures: the playlist and the library.
- Check: test sections 1 and 2 match the expected output.

Step 2. Playing. Write `play` and `playCount`.

- Write one private helper that makes a song current and adds 1 to its play count. Use it everywhere a song starts.
- Check: the first two lines of test section 3.

Step 3. Up next. Write `queueLater`, `queueNext` and `showUpNext`.

- Check: the first five lines of test section 4.

Step 4. Next. Write `next`.

- Handle up next first, then the playlist, then the "nothing left" case.
- Check: every "next" and "now playing" line in test sections 3 and 4.

Step 5. Back. Write `back` and `showHistory`.

- Check: the "history" lines in sections 3 and 4, and all of section 5.

Step 6. Remove. Write `removeSong`, then go back and make `next`, `back`, `showUpNext` and `showHistory` skip removed songs.

- Check: test sections 6 and 7.

Step 7. Finish.

- Run the full test. Sections 8 and 9 check play counts and an empty player.
- Compare your output to `expected-output.txt` line by line.
- Fill in the Big-O table and write your two justifications (see What to submit).

Hints

Design hints (both languages)

1. **Store the node, not just the song.** If you build the playlist, make the library map id → playlist node. Then `removeSong` and "the song after the current one" are both O(1): look up the node, then use `node.next` and `node.prev`. If the map stores only the song, you have to scan the playlist to find it, which is O(n).
2. **One way in.** Write a single private helper, for example `start(song)`, that sets `current` and adds 1 to the play count. Call it from `play`, `next` and `back`. This keeps the count correct with one line of code.
3. **Decide first, change second.** In `next()` and `back()`, find the song you are moving to before you touch `current` or history. If you find nothing, return `null` and nothing has changed.
4. **Skipping removed songs is a loop.** While up next is not empty: take the front song; if the library still has its id, use it and stop; otherwise keep going. `back()` does the same with history.
5. **The current song is always in the library.** You are not allowed to remove it, so looking it up to find the next playlist song is always safe.
6. **`back()` does not push onto history.** If it did, pressing back twice would bounce between the same two songs.
7. **Removing a node has four cases.** The node is the only one, the head, the tail, or in the middle. Draw each case on paper and check `head` and `tail` afterward.

8. **The show methods filter.** `showUpNext()` and `showHistory()` only list songs whose id is still in the library.

JavaScript hints

- `Map` basics: `map.set(id, value)`, `map.get(id)`, `map.has(id)`, `map.delete(id)`.
- `map.get(id)` returns `undefined` for a missing key. Return `null` yourself when the spec says `null`.
- Play count in one line: `map.set(id, (map.get(id) || 0) + 1)`.
- Reuse path: an array works as a stack with `push` and `pop`, both O(1). As a deque, `push` is O(1) but `shift` and `unshift` are O(n). Put that in your Big-O table.
- To list a linked structure, walk it with a loop: `for (let n = head; n !== null; n = n.next)`.

Java hints

- Declare with the interface type: `Map<String, Node> library = new HashMap<>();` and `Deque<Song> upNext = new ArrayDeque<>();`.
- `ArrayDeque` as a deque: `addFirst`, `addLast`, `pollFirst`. As a stack: `push`, `pop`, `peek`.
- `pollFirst()` returns `null` on an empty deque. `pop()` and `removeFirst()` throw an exception. Check `isEmpty()` first.
- An `ArrayDeque` used as a stack iterates from the top down, so a for-each loop already gives "most recent first".
- Play count in one line: `playCounts.merge(id, 1, Integer::sum)`. Read it with `playCounts.getOrDefault(id, 0)`.
- Compare ids with `.equals()`, not `==`.
- Reuse path: `LinkedList.remove(Object)` and `indexOf` are O(n) because they scan. Put that in your Big-O table.

Common bugs

What you see	Likely cause
A play count is one too low	`next()` or `back()` sets `current` without going through your helper
"next at end" prints `null` but now playing changed	You changed `current` or history before checking that a next song exists
History grows when you press back	`back()` is pushing onto history
A removed song still shows in up next	`showUpNext()` is not filtering by the library
`next()` plays a removed song	The skip loop is missing, or it checks the wrong map
Crash on an empty player	`next()` or `back()` uses `current` without checking for `null`

What you see	Likely cause
Playlist loses songs after a remove	`head` or `tail` was not updated in one of the four cases

Test script and expected output

The starter folder includes a test script and `expected-output.txt`. Your output must match exactly. Do not edit the test script.

- JavaScript: `node test.js`
- Java: `javac *.java && java Test`

Both languages print the same thing:

```
--- 1. Build the playlist ---
true
true
true
true
true
true
duplicate s3: false
playlist: [Night Drive, Slow Tide, Copper Sky, Static Bloom, Low Orbit, Glass Rivers]
--- 2. Look up songs ---
find s4: Static Bloom
find s9: null
--- 3. Play and move forward ---
play s1: Night Drive
play s9: null
next: Slow Tide
next: Copper Sky
history: [Slow Tide, Night Drive]
--- 4. Up next ---
true
true
true
queue s9: false
up next: [Night Drive, Low Orbit, Glass Rivers]
next: Night Drive
now playing: Night Drive
history: [Copper Sky, Slow Tide, Night Drive]
--- 5. Back button ---
back: Copper Sky
up next: [Night Drive, Low Orbit, Glass Rivers]
history: [Slow Tide, Night Drive]
--- 6. Remove songs ---
remove s5: true
remove current (s3): false
remove s9: false
playlist: [Night Drive, Slow Tide, Copper Sky, Static Bloom, Glass Rivers]
up next: [Night Drive, Glass Rivers]
--- 7. Removed songs are skipped ---
```

```
next: Night Drive
next: Glass Rivers
next at end: null
now playing: Glass Rivers
back: Night Drive
up next: [Glass Rivers]
--- 8. Play counts ---
s1: 4
s3: 2
s6: 1
s5 (removed): 0
--- 9. Empty player ---
next: null
back: null
now playing: null
playlist: []
```

Trace section 7 by hand before you code it. Low Orbit (s5) was removed in section 6 but is still inside up next, so the second `next` skips it and plays Glass Rivers.

Extra credit

Each feature is worth 5 points. You can earn at most 10. Extra features must not change the required output, so test them in a separate file.

Feature	What to add	Structure to think about
Build everything	Write the playlist, deque and stack yourself from linked nodes, with no arrays or library collections behind them	Linked nodes
Undo	`undo()` reverses the last `addSong` or `removeSong`	A second stack of actions
Repeat mode	`setRepeat(true)` makes `next()` wrap from the last playlist song to the first	Circular linked list, or a wrap check
Recently played	`recentlyPlayed()` returns the last 10 different songs, newest first	A deque that drops its oldest item when full
Top songs	`topSongs(n)` returns the n most played titles	The play-count map, then a sort
Shuffle	`shuffleUpNext()` fills up next with every playlist song in random order, leaving the playlist unchanged	Copy to an array, shuffle, load the deque

What to submit

Submit three things by [due date] through [submission method].

1. **Your code.** JavaScript: `musicPlayer.js`. Java: `MusicPlayer.java`, plus any extra class files you wrote.
2. **Your test output.** Copy what the test script printed into `my-output.txt`.
3. **A short write-up** (one page, PDF or text) with the two parts below.

Part A: Big-O table

Copy this table and fill in every row for your own code.

Method	Structures it touches	Big-O	One-line reason
`addSong`			
`findSong`			
`play`			
`queueLater`			
`queueNext`			
`next`			
`back`			
`removeSong`			
`showPlaylist`			

Part B: Defend two choices

Pick two of your five structures. For each, write 3 to 5 sentences that answer:

- What job does it do in the player?
- What would get slower or harder if you swapped it for a plain array?
- If you reused a built-in, what does it cost compared with building your own?

Write it the way you would say it in an interview. "I used a deque because both ends are O(1)" is stronger than "I used a deque because it seemed right."